Laravel

Creación de Vistas en Laravel y uso de Bladecd

Pepe Devesa

ies SERPIS | Departament Informàtica

Continguts

1 Vistas con Blade **3**
1.1 Ventajas principales de Blade 3

2 Pasar valores a las Vistas **4**
2.1 Sobre los enlaces a otras rutas 5

3 Creación de páginas HTML completas **6**
3.1 Definir las rutas 9

4 Creación de un Layout base **9**

5 Añadir un menú de navegación **11**

6 Uso de Partials **12**

7 Primeros pasos con el motor de plantillas Blade **13**
7.1 Mostrar un dato solo si existe 14
7.2 Comentarios 14

8 Estructuras de control de flujo en Blade **14**

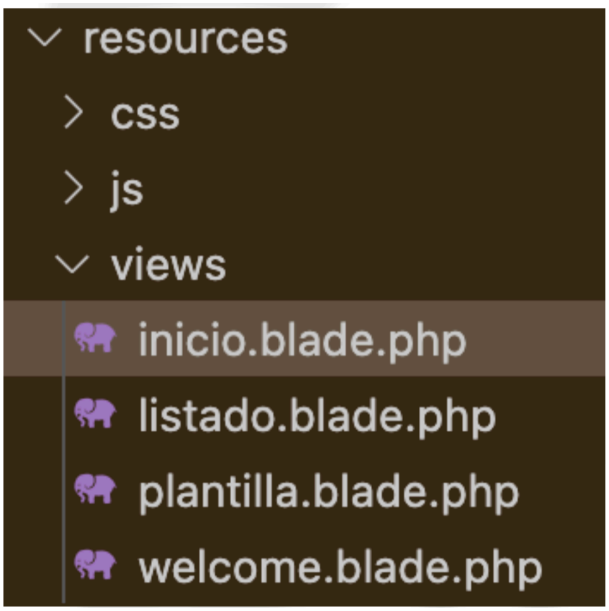
9 Vistas para páginas de error **18**

1 Vistas con Blade

Hasta ahora las rutas que hemos definido devuelven un texto simple, salvo la que ya estaba creada por defecto en el proyecto, que apuntaba a la página de inicio. Si quisiéramos devolver contenido HTML, una opción (costosa) sería devolver dicho contenido generado desde el propio método de la ruta, a través de la instrucción return, pero en lugar de hacer esto desde dentro de la propia función de respuesta, lo más habitual (y recomendable) es generar una vista con el contenido HTML que se quiere enviar al cliente.

La forma general de mostrar vistas en Laravel es hacer que las rutas devuelvan (`return`) una determinada vista. Para ello, se puede emplear la función `view` de Laravel, indicando el nombre de la vista a generar o mostrar.

Por defecto, en la carpeta `resources/views` tenemos disponible una vista de ejemplo llamada `welcome.blade.php` . Es la que se utiliza como página de inicio en la ruta raíz en routes/web.php :



(a) vistas

1.1 Ventajas principales de Blade

- Permite reutilizar código con layouts y partials.
- Tiene una sintaxis muy clara y elegante.
- Es más rápido y organizado que escribir HTML puro.
- Permite incluir recursos estáticos (árbol de carpetas, CSS, imágenes) fácilmente.

```
Route::get('/', function () {
    return view('welcome');
});
```

Notar que **no** es necesario indicar el path o ruta hacia el archivo de la vista, ni tampoco la extensión, puesto que Laravel asume que por defecto las vistas se encuentran en la carpeta `resources/views`, con la extensión `.blade.php` (que hace referencia al motor de plantillas Blade que veremos a continuación), o simplemente con extensión .php (en el caso de vistas simples que no utilicen Blade).

Podemos, por ejemplo, crear una vista sencilla dentro de esta carpeta de vistas (llamémosla inicio.blade.php), con un contenido HTML básico:

```
<html>
   <head>
      <title>Inicio</title>
   </head>
   <body>
      <h1>Página de inicio</h1>
   </body>
</html>
```

Y podemos utilizar esta vista como página de inicio:

```
Route::get('/', function () {
    return view('inicio');
});
```

2 Pasar valores a las Vistas

Es muy habitual pasar cierta información a ciertas vistas, como por ejemplo:

- listados de datos a mostrar.
- datos de un elemento en concreto.

Por ejemplo, si queremos dar un mensaje de bienvenida a un nombre (supuestamente variable), debemos almacenar el nombre en una variable en la ruta, y pasárselo a la vista al cargarla. Esto puede hacerse, por ejemplo, con el método `with` tras generar la vista, indicando el nombre con que lo vamos a asociar a la vista, y el valor (variable) asociado a dicho nombre. En nuestro caso quedaría así:

```
Route::get('/', function() {
 $nombre = "Pepe";
 return view('inicio')->with('nombre', $nombre);
});
```

Posteriormente, en la vista, deberemos mostrar el valor de esta variable en algún lugar del código HTML.

Podemos emplear PHP tradicional para recoger esta variable:

```
<html>
   <head>
      <title>Inicio</title>
   </head>
   <body>
      <h1>Página de inicio</h1>
      <p>Bienvenido/a <?php echo $nombre; ?></p>
   </body>
</html>
```

Pero es más habitual y limpio emplear una sintaxis específica de Blade, como veremos a continuación. Como alternativas al uso de `with` comentado antes, también podemos utilizar un `array asociativo` (asignando así varios nombres a varios valores):

```
return view('inicio')->with(['nombre' => $nombre, ...]);
```

Asimismo, podemos utilizar este mismo array como segundo parámetro de la función `view` , y prescindir así de `with`:

```
return view('inicio'),['nombre' => $nombre, ...]);
```

Y también podemos utilizar una función llamada `compact` como segundo parámetro de view. A esta función le pasamos únicamente el nombre de la variable que usaremos en la vista y, **siempre que la variable asociada se llame igual**, establece la asociación por nosotros:

```
return view('inicio', compact('nombre'));
```

La función `compact` admite tantos parámetros como datos queramos enviar a la vista por separado, cada uno con su nombre asociado. Si simplemente vamos a devolver una vista con poca información asociada, o poca lógica interna, también podemos abreviar el código anterior llamando directamente a `view` , en lugar de a `get` primero, en el archivo `routes/web.php` , y le pasamos así la información asociada a la vista:

```
Route::view('/', 'inicio', ['nombre' => 'Pepe']);
```

2.1 Sobre los enlaces a otras rutas

Hemos comentado brevemente en puntos anteriores que, gracias a Blade y a los nombres en las rutas, podemos enlazar una vista con otra de dos formas:

de forma tradicional…

```
echo '<a href="/contacto">Contacto</a>';
```

… o bien empleando la función route seguida del nombre que le hemos dado a la ruta:

```
<a href="{{ route('ruta_contacto') }}">Contacto</a>
```

Por ejemplo, podemos poner un enlace a la vista del listado de libros que hemos creado antes (y que hemos nombrado como listado_libros de este modo en nuestra vista de `inicio.blade.php`:

```
<p>Bienvenido/a {{ $nombre }}</p>
<p><a href="{{ route('listado_libros') }}">Listado de libros</a></p>
```



(b) Enlace

3 Creación de páginas HTML completas

Vamos a comenzar por crear varias páginas HTML básicas con todo su código HTML. Más adelante veremos cómo optimizar el código usando layouts, partials y componentes.

Vamos a crear 4 páginas básicas en resources/views/landing/:

- **index.blade.php**: página de inicio
- **about.blade.php**: Página típica about, datos de la empresa
- **services.blade.php**: Página con los servicios que ofrece la empresa
- **contact.blade.php**: Página de contacto

Las vistas se encontraran ubicadas en `resources/views/landing/` y tienen la extensión .blade.php.



(c) landing

index.blade.php

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Inicio</title>
</head>
<body>
    <header>
        <h1>Bienvenido a nuestra web</h1>
    </header>

    <main>
        <p>Esta es la página de inicio.</p>
    </main>

    <footer>
        <p>Pie de página &copy; 2025</p>
    </footer>
</body>
</html>
```

about.blade.php

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Sobre nosotros</title>
</head>
<body>
    <header>
        <h1>Sobre nosotros</h1>
    </header>

    <main>
```

```
        <p>Información sobre nuestra empresa.</p>
    </main>

    <footer>
        <p>Pie de página &copy; 2025</p>
    </footer>
</body>
</html>
```

services.blade.php

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Servicios</title>
</head>
<body>
    <header>
        <h1>Servicios</h1>
    </header>

    <main>
        <p>Descripción de nuestros servicios.</p>
    </main>

    <footer>
        <p>Pie de página &copy; 2025</p>
    </footer>
</body>
</html>
```

contact.blade.php

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Contacto</title>
</head>
<body>
    <header>
        <h1>Contacto</h1>
    </header>

    <main>
        <p>Formulario de contacto.</p>
    </main>

    <footer>
        <p>Pie de página &copy; 2025</p>
    </footer>
</body>
</html>
```

3.1 Definir las rutas

Ahora vamos a definir las rutas para que las vistas tengan su punto de entrada. Vamos a `routes/web.php` y definimos las rutas para cada página usando `Route::view()`, que es una forma sencilla de devolver una vista sin necesidad de un controlador.

```
Route::view('/', 'landing.index')->name('home');
Route::view('/about', 'landing.about')->name('about');
Route::view('/services', 'landing.services')->name('services');
Route::view('/contact', 'landing.contact')->name('contact');
```

y las probamos en el navegador.

4 Creación de un Layout base

Podemos darnos cuenta que todas las páginas comparten más del 90% de su estructura HTML, y eso es **poco eficiente**. Cualquier cambio en la cabecera o pie de página, tendríamos que hacerlo en cada archivo. Para ello `blade` nos permite crear un `layout` base que contenga la estructura común de las páginas, y que pueda ser reutilizado en todas las vistas.

Los layouts son plantillas que contienen la estructura común de las páginas. En Laravel, los layouts se crean en la carpeta **resources/views/landing/layouts/**.

landing.blade.php

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>
        @yield('title')
    </title>
</head>
<body>
    <header>
        @yield('header')
    </header>

    <main>
        @yield('content')
    </main>

    <footer>
        <p>Pie de página &copy; 2025</p>
    </footer>
</body>
</html>
```

- **@yield('title')** es un marcador de posición que se reemplazará por el contenido de la sección title en las vistas que extiendan este layout.
- **@yield('header')** y @yield('content') son marcadores de posición para el contenido específico de cada vista.

Modificar las vistas para que extiendan el layout:

index.blade.php

```
@extends('landing.layouts.landing')
@section('title', 'Inicio')
@section('header')
    <h1>Bienvenido a nuestra web</h1>
@endsection
@section('content')
    <p>Esta es la página de inicio.</p>
@endsection
```

about.blade.php

```
@extends('landing.layouts.landing')
@section('title', 'Sobre nosotros')
@section('header')
    <h1>Sobre nosotros</h1>
@endsection
```

```
@section('content')
    <p>Información sobre nuestra empresa.</p>
@endsection
```

services.blade.php

```
@extends('landing.layouts.landing')
@section('title', 'Servicios')
@section('header')
    <h1>Servicios</h1>
@endsection
@section('content')
    <p>Descripción de nuestros servicios.</p>
@endsection
```

contact.blade.php

```
@extends('landing.layouts.landing')
@section('title', 'Contacto')
@section('header')
    <h1>Contacto</h1>
@endsection
@section('content')
    <p>Formulario de contacto.</p>
@endsection
```

5 Añadir un menú de navegación

Ahora vamos a poner en práctica lo aprendido anteriormente. Vamos a añadir un menú a nuestra páginas. Como en todas las páginas el menú es el mismo, lo añadiremos al layout. De esta manera vemos que añadieno el menú en un solo logar nos vale para todas las páginas. Solo hay que mantener un menú, siento insistir en lo mismo pero es muy importante.

En el layout, dentro del `header`, incluimos el menú de navegación:

Para ello, vamos a modificar la plantilla y vamos a añadir un menú de navegación:

```
<header>
    <nav>
        <ul>
            <li><a href="/">Inicio</a></li>
            <li><a href="/about">Sobre nosotros</a></li>
            <li><a href="/services">Servicios</a></li>
            <li><a href="/contact">Contacto</a></li>
        </ul>
    </nav>
    @yield('header')
```

```
</header>
```

Podemos probarlo y puede funionar. Pero no es la mejor forma de hacerlo. Puede dar problemas, por ejemplo, si cambiamos la URL de una de las páginas, tendríamos que cambiarlo en todas las vistas. Para evitar esto, Laravel nos permite nombrar las rutas y usar esos nombres para generar los enlaces. En resumen, en lugar de escribir las URLs directamente, usamos el nombre de la ruta

Menú de navegación usando nombres de rutas:

```
<nav>
    <ul>
        <li><a href="{{ route('home') }}">Inicio</a></li>
        <li><a href="{{ route('about') }}">Sobre nosotros</a></li>
        <li><a href="{{ route('services') }}">Servicios</a></li>
        <li><a href="{{ route('contact') }}">Contacto</a></li>
    </ul>
</nav>
```



Buenas prácticas
Es mejor usar `{{ route('nombre')}}` en lugar de URLs escritas a mano, así si la ruta cambia solo lo actualizas en un lugar. De esta forma, si por lo que sea en un futuro hay que modificar la URL de una ruta (por SEO, por modificaciones en la web, etc), solo hay que cambiarla en un único sitio, en la definición de la ruta. Esto ayuda a mantener el código limpio y evitar errores.

6 Uso de Partials

Pero aún podemos mejorar más el código. En un sitio web podemos tener más de una plantilla y el menú repite en todas ellas. Si queremos cambiar el menú, tendríamos que hacerlo en cada layout. Para evitar esto, `blade` nos permite crear `partials`. Utilizaresmos una plantilla llamada `_partials` para guardar fragmentos de código reutilizables, como el menú de navegación. En algunas carpetas laravel aconseja utilizar _ para que siempre estén al principio del listado de archivos.

En **resources/views/_partials/**, **creamos un archivo** nav.blade.php** para el menú:

```
<nav>
    <ul>
        <li><a href="{{ route('home') }}">Inicio</a></li>
        <li><a href="{{ route('about') }}">Sobre nosotros</a></li>
        <li><a href="{{ route('services') }}">Servicios</a></li>
        <li><a href="{{ route('contact') }}">Contacto</a></li>
    </ul>
```

```
</nav>
```

Incluimos el partial en el layout:

```
@include('_partials.nav')
```

Ahora el layout quedará asi:

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>
        @yield('title')
    </title>
</head>
<body>
    <header>
        @include('_partials.nav')
        @yield('header')
    </header>

    <main>
        @yield('content')
    </main>

    <footer>
        <p>Pie de página &copy; 2025</p>
    </footer>
</body>
</html>
```

7 Primeros pasos con el motor de plantillas Blade

Hemos comentado en el apartado anterior que el uso de Blade permite simplificar la sintaxis y la forma de procesar algunas cosas en nuestras vistas Siempre que creemos el archivo de la vista con la extensión `.blade.php`, nos permitirá automáticamente aprovechar la sintaxis y funcionalidades de Blade en nuestras vistas. Por ejemplo, si queremos mostrar el contenido de la variable `nombre` que hemos pasado antes a la página de inicio, en lugar de hacer un rudimentario `echo` en PHP, podemos emplear una sintaxis de `dobles llaves {{ }}`, facilitada por Blade, para mostrar el contenido de esa variable o el contenido de una función. Con esto la línea que mostraba el nombre pasa de ser así…

```
Bienvenido/a <?php echo $nombre ?>
```

…a ser así:

```
Bienvenido/a  {{ $nombre }}
La hora actual es {{ time() }}
```

Blade se encarga de escapar el resultado llamando a `htmlentities` para prevenir errores y ataques de tipo XSS(Cross Site Scripting), es decir, que se inyectan scripts de Javascript con la variable a mostrar. Si en algún caso no queremos escapar los datos tendremos que llamar a:

```
Bienvenido/a  {!! $nombre !!}
```



En general siempre tendremos que usar las llaves dobles, en especial si vamos a mostrar datos que son proporcionados por los usuarios de la aplicación. Esto evitará que inyecten símbolos que produzcan errores o inyecten codigo javascript que se ejecute sin que nosotros queramos. Por lo tanto, este último método solo tenemos que utilizarlo si estamos seguros de que no queremos que se escape el contenido.

7.1 Mostrar un dato solo si existe

Para comprobar que una variable existe o tiene un determinado valor podemos utilizar el operador ternario de la forma:

```
{{ isset($name) ? $name : 'Valor por defecto' }}
```

O simplemente usar la notación que incluye Blade para este fin:

```
{{ $name or 'Valor por defecto' }}
```

7.2 Comentarios

Para escribir comentarios en Blade se utilizan los símbolos **{{- y -}}**, por ejemplo:

```
{{-- Este comentario no se mostrará en HTML --}}
```

8 Estructuras de control de flujo en Blade

Para iterar sobre un conjunto de datos (array), podemos emplear la directiva `@foreach` , con una sintaxis similar al foreach de PHP, pero sin necesidad de llaves. Basta con finalizar el bucle con la directiva `@endforeach` , de este modo:

```
<ul>
   @foreach($elementos as $elemento)
      <li>{{ $elemento }}</li>
   @endforeach
</ul>
```

En el caso de querer realizar alguna comprobación (por ejemplo, si el array anterior está vacío, para mostrar un mensaje pertinente), usamos la directiva `@if` , cerrada por su correspondiente pareja `@endif` . Opcionalmente, se puede intercalar una directiva `@else` para el camino alternativo, o también `@elseif` para indicar otra condición. El ejemplo anterior podría quedar así:

```
<ul>
   @if($elementos)
      @foreach($elementos as $elemento)
         <li>{{ $elemento }}</li>
      @endforeach
   @else
      <li>No hay elementos que mostrar</li>
   @endif
</ul>
```

También podemos comprobar si una variable está definida. En este caso, reemplazamos la directiva `@if por @isset` , con su correspondiente cierre `@endisset`.

```
<ul>
   @isset($elementos)
      @foreach($elementos as $elemento)
         <li>{{ $elemento }}</li>
      @endforeach
   @else
      <li>No hay elementos que mostrar</li>
   @endisset
</ul>
```

Sin embargo, con cualquiera de estas opciones tenemos un problema:

- en el primer caso, si la variable `$elementos` no está definida, mostrará un error de PHP.
- En el segundo caso, si la variable sí está definida pero no contiene elementos, no se mostrará nada por pantalla.

Una tercera estructura alternativa que agrupa estos dos casos (controlar a la vez que la variable esté definida y tenga elementos) es emplear la directiva `@forelse` en lugar de `@foreach` . Esta directiva permite una cláusula adicional `@empty` para indicar qué hacer si la colección no tiene elementos o está sin definir. El ejemplo anterior quedaría ahora así de abreviado:

```
<ul>
```

```
      @forelse($elementos as $elemento)
         <li>{{ $elemento }}</li>
      @empty
        <li>No hay elementos que mostrar</li>
      @endforelse
</ul>
```

En este tipo de iteradores (`@foreach o @forelse`), tenemos disponible un objeto llamado `$loop` , con una serie de propiedades sobre el bucle que estamos iterando, como por ejemplo:

- `index` : posición dentro del array por la que vamos.
- `count` : total de elementos.
- `first y last` : booleanos que determinan si es el primer o último elemento, respectivamente.

Podemos ver todas las propiedades disponibles en este objeto llamando a `var_dump` :

```
<ul>

      @forelse($elementos as $elemento)
         <li>{{ $elemento }}{{var_dump($loop)}}</li>
      @empty
        <li>No hay elementos que mostrar</li>
      @endforelse
</ul>
```

Si, por ejemplo, queremos determinar si es el último elemento de la lista, y mostrar un mensaje o estilo especial, podemos hacer algo como esto:

```
<ul>

      @forelse($elementos as $elemento)
         <li>{{ $elemento }}
             {{$loop->last ? "Ultimo elemento" : "" }}
         </li>
      @empty
        <li>No hay elementos que mostrar</li>
      @endforelse
</ul>
```

Existen otros tipos de estructuras iterativas y selectivas en Blade, como `@while` , `@for o @switch` , entre otras. Podéis consultar sobre su uso en la documentación oficial de Blade.

Vamos a aplicar esto en nuestro ejemplo del proyecto biblioteca. Definiremos una ruta para sacar un listado de libros y, de momento, vamos a crear a *mano* dicho listado en el propio método de enrutamiento, y se lo pasaremos a una vista llamada `listado.blade.php`. Por un lado, la nueva ruta para el listado puede quedar así:

```
Route::get('listado', function() {
   $libros = array(
      array("titulo" => "El juego de Ender",
            "autor" => "Orson Scott Card"),
      array("titulo" => "La tabla de Flandes",
            "autor" => "Arturo Pérez Reverte"),
      array("titulo" => "La historia interminable",
            "autor" =>  "Michael Ende"),
      array("titulo" => "El Señor de los Anillos",
            "autor" => "J.R.R. Tolkien")
            );

      return view('listado', compact('libros'));
      })->name('listado_libros');
```

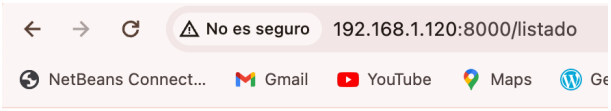
Por su parte, la vista listado.blade.php puede quedar así:

```
@extends('plantilla')

@section('titulo', 'Listado de libros')

@section('contenido')
    <h1>Listado de libros</h1>
    <ul>
    @forelse ($libros as $libro)
        <li>{{ $libro["titulo"] }} ({{ $libro["autor"] }})</li>
    @empty
        <li>No se encontraron libros</li>
    @endforelse
    </ul>
@endsection
```

y el resultado:



Listado de libros

- El juego de Ender (Orson Scott Card)
- La tabla de Flandes (Arturo Pérez Reverte)
- La historia interminable (Michael Ende)
- El Señor de los Anillos (J.R.R. Tolkien)

(d) Listado de libros

9 Vistas para páginas de error

A lo largo de estas sesiones, algunas acciones que hagamos provocarán páginas de error con determinados códigos, como por ejemplo *404* para páginas no encontradas. Si queremos definir el aspecto y estructura de estas páginas, basta con crear la vista correspondiente en la carpeta `resources/views/errors`, por ejemplo, `resources/views/errors/404.blade.php` para el error 404 (anteponemos el código de error al sufijo de la vista).

```
@extends('plantilla')

@section('titulo', 'Error 404')

@section('contenido')
  <h1>Error</h1>
  <p>Documento no encontrado</p>
@endsection
```